

LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

Jobsheet 01 — Pengantar Konsep Pemrograman Berorientasi Objek

Nama : Ringga Budi Utama
NIM : 254107020251
Program Studi : Teknik Informatika
Kelas : Internasional 2I

JURUSAN TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK NEGERI MALANG

2026

DAFTAR ISI

1. Kompetensi	3
2. Pendahuluan	3
2.1 Pemrograman Prosedural vs Berorientasi Objek	3
2.2 Konsep Dasar PBO	5
3. Percobaan	7
3.1 Percobaan 1	7
3.2 Percobaan 2	9
4. Kesimpulan	11
5. Pertanyaan	11
6. Tugas Praktikum	13

1. Kompetensi

Setelah menempuh materi percobaan ini, mahasiswa mampu mengenal:

1. Perbedaan paradigma berorientasi objek dengan paradigma struktural
2. Konsep dasar PBO

2. Pendahuluan

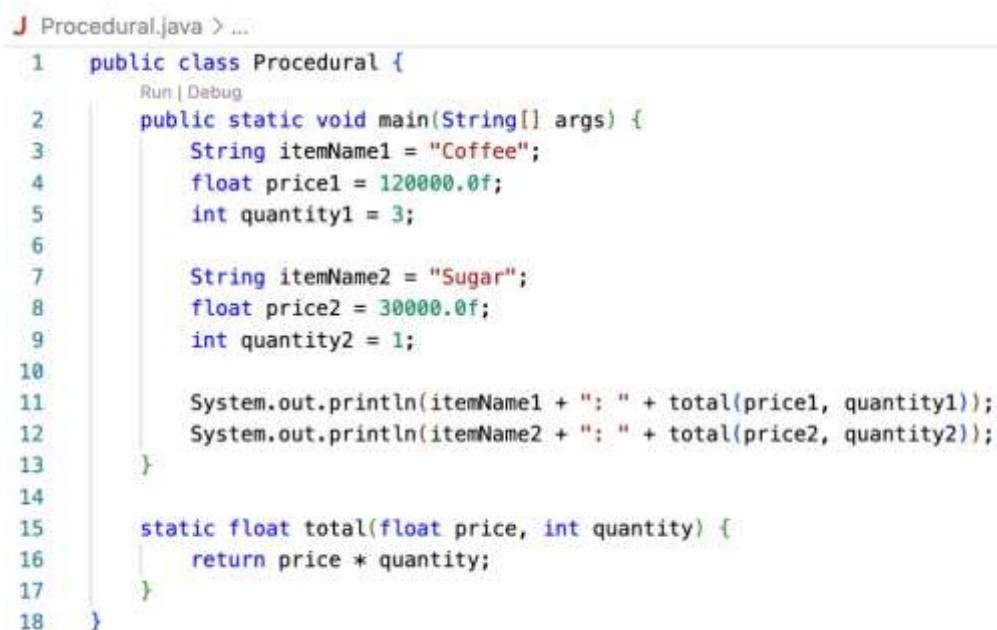
2.1 Pemrograman Prosedural vs Berorientasi Objek

Perbedaan mendasar antara pemrograman prosedural dengan pemrograman berorientasi objek (PBO) adalah: pada pemrograman prosedural, program dipecah ke dalam sub-program atau **fungsi**. Sedangkan pada PBO, program dipecah ke dalam **object**, di mana objek tersebut membungkus **state** dan **method**.

Kelebihan PBO adalah program dapat lebih fleksibel dan modular — jika ada perubahan fitur, maka dapat dipastikan keseluruhan program tidak akan terganggu. Berbeda dengan prosedural, perubahan sedikit fitur saja kemungkinan dapat mengganggu keseluruhan program.

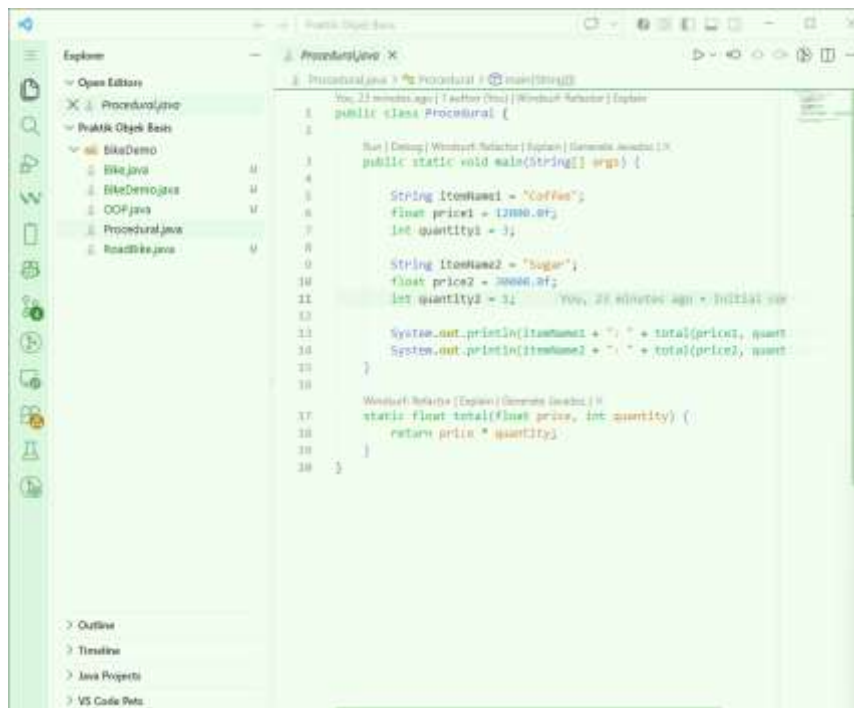
Untuk lebih jelas, berikut perbedaan antara pemrograman prosedural dengan pemrograman berorientasi objek.

- **Procedural Programming**



```

Procedural.java > ...
1  public class Procedural {
2      public static void main(String[] args) {
3          String itemName1 = "Coffee";
4          float price1 = 120000.0f;
5          int quantity1 = 3;
6
7          String itemName2 = "Sugar";
8          float price2 = 30000.0f;
9          int quantity2 = 1;
10
11         System.out.println(itemName1 + ": " + total(price1, quantity1));
12         System.out.println(itemName2 + ": " + total(price2, quantity2));
13     }
14
15     static float total(float price, int quantity) {
16         return price * quantity;
17     }
18 }
  
```

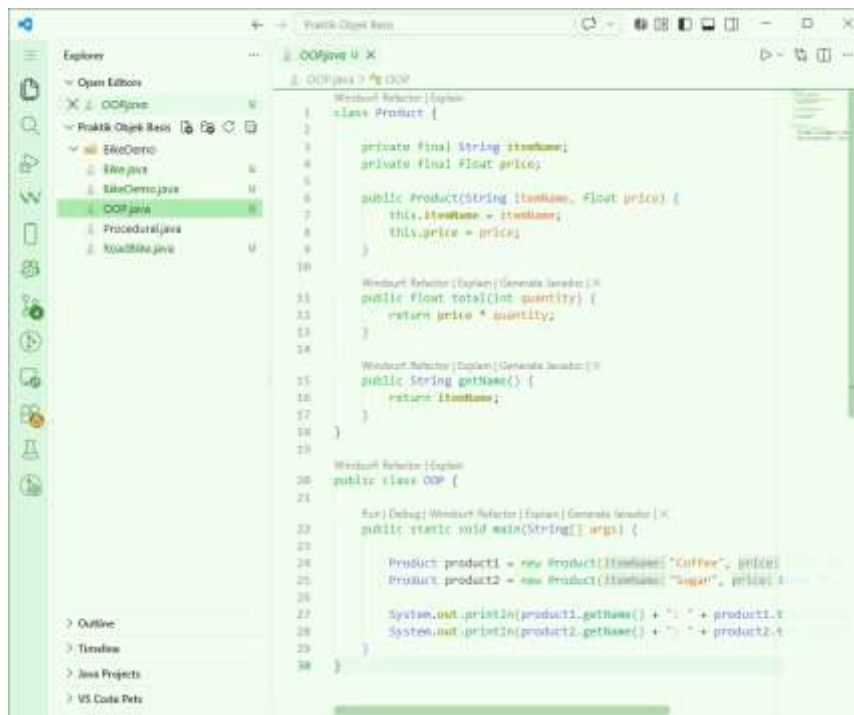


- **Object Oriented Programming**

```

1  class Product {
2
3      private final String itemName;
4      private final float price;
5
6      public Product (String itemName, float price) {
7          this.itemName = itemName;
8          this.price = price;
9      }
10
11     public float total(int quantity) {
12         return price * quantity;
13     }
14
15     public String getName() {
16         return itemName;
17     }
18 }
19
20 public class OOP {
21     public static void main(String[] args) {
22
23         Product product1 = new Product("Coffee", 12000.0f);
24         Product product2 = new Product("Sugar", 30000.0f);
25
26         System.out.println(product1.getName() + ": " + product1.total(3));
27         System.out.println(product2.getName() + ": " + product2.total(1));
28     }
29 }

```



Berdasarkan dua buah potongan kode tersebut, kode program dalam bentuk object lebih tertata dan lebih efisien karena tidak menggunakan kode yang berulang dibandingkan kode prosedural. Hal ini dapat dilihat pada contoh kode program menggunakan object, di mana object baru dibuat menggunakan kata kunci `new` yang diikuti oleh nama kelasnya.

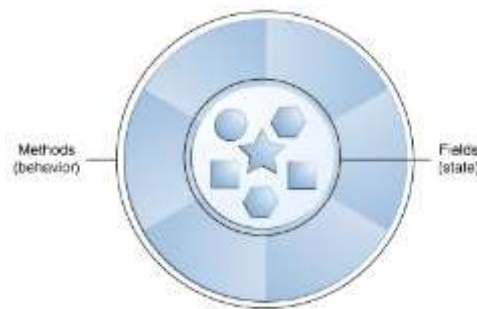
Catatan: Di dalam modul praktikum ini, kita akan mencoba membuat class, membuat object, dan memanggil method-method yang ada di dalam object. Penjelasan mengenai anatomi class (atribut, method) akan dibahas lebih detail di modul praktikum berikutnya.

2.2 Konsep Dasar PBO

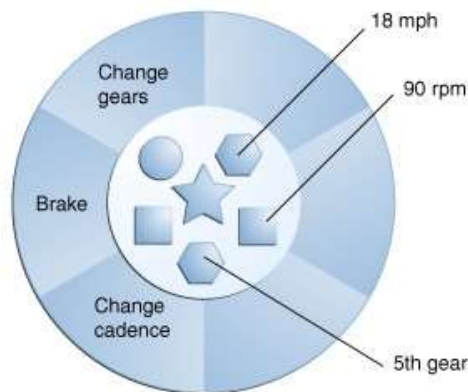
Di dalam PBO dikenal beberapa aspek, yaitu:

a. Object

Object adalah suatu rangkaian dalam program yang terdiri dari state dan behaviour. Object pada software dimodelkan sedemikian rupa sehingga mirip dengan objek yang ada di dunia nyata. Objek memiliki state dan behaviour. State adalah ciri-ciri atau atribut dari objek tersebut — misalnya objek Sepeda, memiliki state brand, speed, gear, dan sebagainya. Sedangkan behaviour adalah perilaku yang dapat dilakukan objek tersebut — misalnya pada Sepeda, behaviour-nya antara lain speed changes, gear changes, brake, dan lain sebagainya.



Gambar 1. Object pada software



Gambar 2. Sepeda yang dimodelkan sebagai object pada software

b. Class

Class adalah blueprint atau prototype dari object. Ambil contoh object sepeda — terdapat berbagai macam sepeda di dunia, dari berbagai merk dan model. Namun semua sepeda dibangun berdasarkan blueprint yang sama, sehingga setiap sepeda memiliki komponen dan karakteristik yang sama. Sepeda yang Anda miliki di rumah adalah hasil instansiasi dari class sepeda.

c. Enkapsulasi

Disebut juga dengan information-hiding. Dalam berinteraksi dengan objek, sering kali kita tidak perlu mengetahui kompleksitas yang ada di dalamnya. Contoh pada sepeda, ketika kita mengganti gear pada sepeda, kita tinggal menekan tuas gear yang ada di grip setang sepeda saja — kita tidak perlu mengetahui bagaimana cara gear berpindah secara teknis.

d. Inheritance

Disebut juga pewarisan. Inheritance memungkinkan kita untuk mengorganisir struktur program dengan natural, serta memperluas fungsionalitas program tanpa harus mengubah banyak bagian program. Contoh di dunia nyata, objek sepeda dapat diturunkan lagi ke model yang lebih luas, misalnya sepeda gunung (mountain bike) dan road bike, di mana masing-masing dapat memiliki komponen tambahan — misalnya sepeda gunung memiliki suspensi

yang tidak dimiliki sepeda biasa. Dalam hal ini, objek mountain bike dan road bike mewarisi objek sepeda.

e. Polimorfisme

Polimorfisme juga meniru sifat objek di dunia nyata, di mana sebuah object dapat memiliki bentuk atau menjelma menjadi bentuk-bentuk lain. Misalnya objek pesawat terbang — objek ini dapat diwariskan menjadi pesawat jet dan pesawat baling-baling. Keduanya memiliki kemampuan untuk menambah kecepatan, namun secara teknis metode penambahan kecepatan antara pesawat jet dengan baling-baling tentu berbeda, karena masing-masing memiliki jenis mesin yang berbeda.

3. Percobaan

3.1 Percobaan 1

Di dalam percobaan ini, kita akan mendemonstrasikan bagaimana membuat class, membuat object, kemudian mengakses method di dalam class tersebut.

1. Buka Netbeans atau editor Java lain, lalu buat project BikeDemo.
2. Buat class Bike. Klik kanan pada package **BikeDemo** → New → Java Class.
3. Ketikkan kode class Bike di bawah ini.

```

1 public class Bike{
2     private String brand;
3     private int speed;
4     private int gear=1;
5     // Gear 1: max 5 km/h, Gear 2: max 10 km/h, ... Gear 6: max 60 km/h
6     private final int[] GEAR_SPEED_LIMITS = {5, 10, 25, 30, 40, 60};
7     public void setBrand(String brandName){
8         brand = brandName;
9     }
10    public void gearChanges(int gearValue){
11        if (gearValue < 1 || gearValue > 6) {
12            System.out.println("Invalid gear value. Gear must be between 1 and 6.");
13        }
14        else{
15            gear = gearValue;
16        }
17    }
18    public int speedAcceleration(int increment) {
19        speed += increment;
20        if (speed > GEAR_SPEED_LIMITS[gear - 1]) {
21            speed = GEAR_SPEED_LIMITS[gear - 1];
22        }
23        return speed;
24    }
25    public int speedDeceleration(int decrement) {
26        speed -= decrement;
27        if (speed < 0) {
28            speed = 0;
29        }
30        return speed;
31    }
32    public void printInfo(){
33        System.out.println("Brand : " + brand);
34        System.out.println("Speed : " + speed);
35        System.out.println("Gear : " + gear);
36    }
37 }

```

4. Kemudian pada class main, ketikkan kode berikut ini.

```

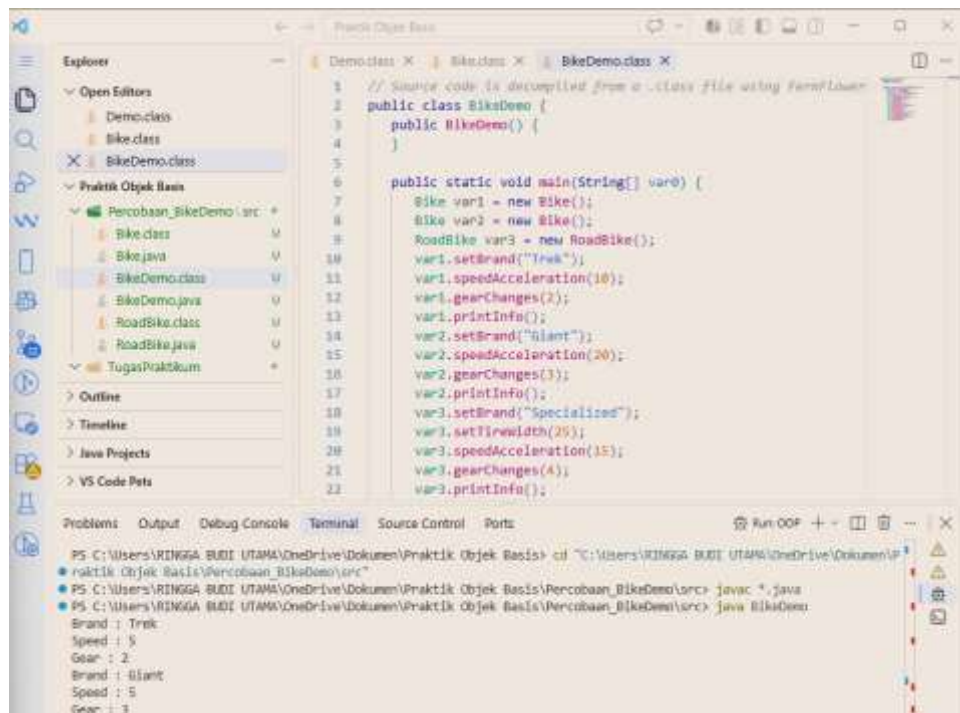
1 public class BikeDemo{
2     public static void main(String[] args){
3         Bike mountainBike1 = new Bike();
4         Bike mountainBike2 = new Bike();
5
6         mountainBike1.setBrand("Trek");
7         mountainBike1.speedAcceleration(10);
8         mountainBike1.gearChanges(2);
9         mountainBike1.printInfo();
10
11        mountainBike2.setBrand("Giant");
12        mountainBike2.speedAcceleration(20);
13        mountainBike2.gearChanges(3);
14        mountainBike2.printInfo();
15    }
16 }

```

```

Brand : Trek
Speed : 5
Gear : 2
Brand : Giant
Speed : 5
Gear : 3

```



3.2 Percobaan 2

Di dalam percobaan ini, akan didemonstrasikan salah satu fitur paling penting dalam PBO, yaitu inheritance. Di sini kita akan membuat class RoadBike untuk kelas sepeda balap yang merupakan turunan/warisan dari class Bike. Pada dasarnya class RoadBike sama dengan class Bike, hanya saja pada road bike terdapat lebar ban. Untuk itu kita tidak perlu membuat class RoadBike dari nol, tetapi cukup mewariskan class Bike ke class RoadBike.

Penjelasan lebih detail tentang inheritance akan dibahas pada modul selanjutnya.

1. Masih pada project BikeDemo, buat class RoadBike.
2. Tambahkan kode **extends Bike** pada deklarasi class RoadBike. Kode extends ini menandakan bahwa class RoadBike mewarisi class Bike.

```
1 public class RoadBike extends Bike {
2
3 }
```

3. Kemudian lengkapi kode RoadBike seperti berikut ini:

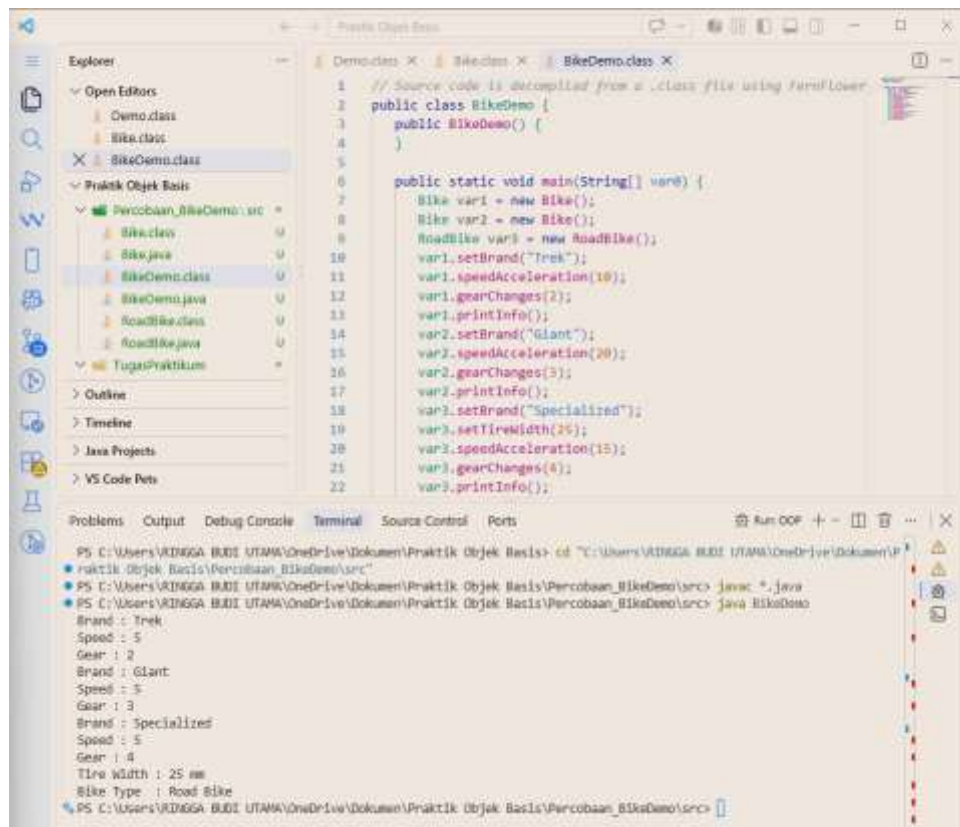
```

1  public class RoadBike extends Bike {
2      private int tireWidth;
3
4      public void setTireWidth(int width) {
5          tireWidth = width;
6      }
7
8      @Override
9      public void printInfo() {
10         super.printInfo();
11         System.out.println("Tire Width : " + tireWidth + " mm");
12         System.out.println("Bike Type : Road Bike");
13     }
14 }

1  public class BikeDemo{
2      public static void main(String[] args){
3          Bike mountainBike1 = new Bike();
4          Bike mountainBike2 = new Bike();
5          RoadBike roadBike1 = new RoadBike();
6
7          mountainBike1.setBrand("Trek");
8          mountainBike1.speedAcceleration(10);
9          mountainBike1.gearChanges(2);
10         mountainBike1.printInfo();
11
12         mountainBike2.setBrand("Giant");
13         mountainBike2.speedAcceleration(20);
14         mountainBike2.gearChanges(3);
15         mountainBike2.printInfo();
16
17         roadBike1.setBrand("Specialized");
18         roadBike1.setTireWidth(25);
19         roadBike1.speedAcceleration(15);
20         roadBike1.gearChanges(4);
21         roadBike1.printInfo();
22     }
23 }

```

4. Cocokkan hasilnya:



4. Kesimpulan

Dari percobaan di atas, kita telah mendemonstrasikan bagaimana paradigma pemrograman berorientasi objek dan mengimplementasikannya ke dalam program sederhana. Kita juga telah mendemonstrasikan salah satu fitur paling penting dari PBO yaitu inheritance, dalam hal ini melalui pembuatan class RoadBike.

Kita mengetahui bahwa RoadBike pada dasarnya sama dengan Bike (memiliki gear, memiliki kecepatan, dapat menambah kecepatan, dapat mengerem, dan pindah gigi), namun ada fitur tambahan yaitu lebar ban. Maka kita tidak perlu membuat class RoadBike dari nol kita cukup extends atau mewariskan dari class Bike, kemudian tinggal menambahkan fitur yang sebelumnya belum ada di class Bike. Inilah salah satu kelebihan PBO yang tidak dimiliki pemrograman prosedural.

5. Pertanyaan

1. Jelaskan perbedaan antara object dengan class!

Menurut saya, class dapat diibaratkan sebagai blueprint atau cetakan yang digunakan untuk menentukan atribut dan method yang nantinya dimiliki oleh sebuah object. Sedangkan object merupakan hasil dari pembuatan atau instansiasi sebuah class menggunakan kata kunci new.

Satu class dapat digunakan untuk membuat banyak object, dan setiap object bisa memiliki nilai atribut yang berbeda-beda. Contohnya, Bike merupakan class yang menjadi cetakan, kemudian dari class tersebut dapat dibuat object seperti mountainBike1 dan mountainBike2. Walaupun berasal dari class yang sama, kedua object tersebut tetap bisa memiliki nilai atribut yang berbeda sesuai dengan kondisi masing-masing.

2. Jelaskan alasan gear dan brand dapat menjadi atribut dari object Bike!

Menurut saya, gear dan brand termasuk state karena keduanya merupakan ciri-ciri atau karakteristik yang dimiliki oleh setiap sepeda. Nilai dari brand dan gear juga bisa berbeda pada setiap object Bike — misalnya, object pertama memiliki brand Trek dengan gear 2, sedangkan object lainnya memiliki brand Giant dengan gear 3.

Oleh karena itu, gear dan brand lebih cocok dijadikan atribut atau variabel di dalam class. Keduanya berfungsi untuk menyimpan data atau nilai dari suatu object, bukan melakukan suatu aksi seperti halnya behaviour atau method.

3. Sebutkan salah satu kelebihan utama dari pemrograman berorientasi objek dibandingkan dengan pemrograman prosedural!

Menurut saya, PBO lebih modular dan fleksibel dibandingkan dengan pemrograman prosedural. Hal ini karena setiap bagian program dapat dikelompokkan dan dibungkus ke dalam class dan object masing-masing. Jadi, ketika ingin menambahkan fitur baru, misalnya menggunakan inheritance, perubahan tersebut tidak harus mengubah keseluruhan program.

Sedangkan pada pemrograman prosedural, program lebih bergantung pada fungsi-fungsi yang saling berhubungan, sehingga ketika dilakukan perubahan pada satu fungsi, ada kemungkinan bagian lain yang menggunakan fungsi tersebut ikut terpengaruh. Menurut saya, inilah yang membuat PBO lebih mudah dikembangkan dan dipelihara ketika program sudah semakin besar.

4. Apakah diperbolehkan melakukan pendefinisian dua buah atribut dalam satu baris kode seperti "public String nama, alamat;"?

Menurut saya, penulisan seperti "public String nama, alamat;" diperbolehkan karena secara sintaks Java kedua variabel tersebut akan memiliki tipe data String. Namun, penulisan tersebut kurang disarankan jika dilihat dari segi kerapian dan keterbacaan kode.

Menurut saya, akan lebih baik jika setiap variabel ditulis pada baris yang berbeda, seperti public String nama; dan public String alamat; pada baris terpisah. Dengan cara tersebut,

kode menjadi lebih mudah dibaca, dan ketika ingin melakukan perubahan atau pengecekan terhadap variabel tertentu, akan lebih mudah untuk dipahami.

5. Pada class RoadBike, jelaskan alasan atribut brand, speed, dan gear tidak lagi ditulis di dalam class tersebut!

Menurut saya, RoadBike extends Bike berarti class RoadBike mewarisi atribut dan method yang dapat diakses dari class Bike sebagai parent class. Jadi, atribut seperti brand, speed, dan gear yang sudah ada di Bike tidak perlu ditulis atau dibuat ulang di dalam RoadBike.

Hal ini merupakan penerapan konsep inheritance dalam PBO. RoadBike cukup menambahkan atribut atau method baru yang memang belum dimiliki oleh Bike, contohnya tireWidth. Dengan cara ini, kode menjadi lebih efisien, lebih rapi, dan dapat mengurangi penulisan kode yang sama secara berulang.

6. Tugas Praktikum

Lakukan langkah-langkah berikut supaya tugas praktikum yang dikerjakan tersistematis:

- a. Foto 4 buah objek di sekitar kalian, dengan 2 objek di antaranya merupakan objek yang mengandung konsep pewarisan (inheritance). Contoh: kulkas, kursi, meja ruang tamu, meja belajar — sehingga diketahui meja ruang tamu dan meja belajar mewarisi objek meja.
- b. Lakukan pengamatan terhadap 4 objek tersebut untuk menentukan atribut dan method-nya.
- c. Berdasarkan 4 buah objek tersebut, buat class-nya dalam bahasa pemrograman Java.
- d. Perlu diperhatikan bahwa terdapat dua class hasil pewarisan, sehingga perlu menambah satu class baru sebagai class yang mewarisi dua class tersebut.
- e. Tambahkan dua atribut untuk setiap class.
- f. Tambahkan tiga method untuk setiap class, termasuk method cetak informasi.
- g. Tambahkan satu class Demo sebagai main.
- h. Instansiasikan satu buah objek untuk setiap class.
- i. Terapkan setiap method untuk setiap objek yang dibuat.
- j. Contoh yang telah disebutkan pada poin 1.a tidak diperbolehkan dipakai dalam pengerjaan tugas praktikum ini.

LAPORAN TUGAS PRAKTIKUM

Jobsheet 01 – Pengantar Konsep Pemrograman Berorientasi Objek (PBO)

Nama : Ringga Budi Utama
NIM : 254107020251
Kelas / Prodi : 2I / Teknik Informatika
Tanggal Praktikum : 27 Agustus 2026

A. Objek yang Diamati

Objek yang diamati dalam tugas praktikum ini ada 4 buah, yaitu kulkas, meja belajar, laptop, dan handphone. Dari keempat objek tersebut, yang mengandung konsep pewarisan (inheritance) adalah laptop dan handphone, karena keduanya sama-sama termasuk kategori perangkat elektronik sehingga dapat diturunkan dari satu class induk yang sama. Sementara itu, kulkas dan meja belajar diperlakukan sebagai objek yang berdiri sendiri (mandiri), karena tidak memiliki kedekatan konsep yang cukup untuk diwariskan dari class yang sama.





B. Pengamatan Atribut dan Method

Berdasarkan pengamatan terhadap keempat objek, diperoleh ciri (atribut/state) dan perilaku (method/behaviour) sebagai berikut:

- **Kulkas** — Memiliki ciri seperti merk dan kapasitas (liter). Perilakunya antara lain mendinginkan isi di dalamnya dan membuka/menutup pintu.
- **Meja Belajar** — Memiliki ciri seperti bahan pembuatnya dan jumlah laci. Perilakunya antara lain menaruh barang di atasnya dan dibersihkan.
- **Laptop** — Selain ciri umum perangkat elektronik (merk, tahun produksi), memiliki ciri khusus seperti kapasitas RAM dan jenis processor. Perilakunya antara lain membuka aplikasi dan menginstall software.
- **Handphone** — Memiliki ciri umum perangkat elektronik yang sama dengan laptop, ditambah ciri khusus seperti kapasitas baterai dan jumlah kamera. Perilakunya antara lain menelepon dan mengirim pesan.

C & D. Rancangan Class dan Struktur Pewarisan

Karena laptop dan handphone sama-sama merupakan perangkat elektronik, dibuat satu class baru bernama Elektronik yang menampung atribut dan method yang dimiliki bersama oleh keduanya (merk, tahun produksi, nyalakan, matikan, cetak info). Class Laptop dan HandPhone kemudian dibuat dengan extends Elektronik, sehingga keduanya tidak perlu menuliskan ulang atribut yang sama dan cukup menambahkan atribut/method yang menjadi ciri khas masing-masing. Sedangkan class Kulkas dan MejaBelajar dibuat sebagai class mandiri, tanpa hubungan pewarisan dengan class lain.

```
Elektronik  (class baru, induk / parent)
|
+-- Laptop    extends Elektronik
+-- HandPhone extends Elektronik
```

```
Kulkas      (class mandiri)
MejaBelajar (class mandiri)
```

Struktur pewarisan class pada tugas praktikum

E & F. Atribut dan Method Tiap Class

Setiap class ditambahkan minimal dua atribut baru dan tiga method baru (termasuk method cetak informasi), sesuai ketentuan pada jobsheet:

Class	Atribut (baru)	Method (termasuk cetak info)
Elektronik	merk, tahunProduksi	nyalakan(), matikan(), cetakInfo()
Laptop	kapasitasRAM, jenisProcessor	bukaAplikasi(), installSoftware(), cetakInfo() (override)
HandPhone	kapasitasBaterai, jumlahKamera	teleponSeseorang(), kirimPesan(), cetakInfo() (override)
Kulkas	merk, kapasitas	dinginkan(), bukaPintu(), cetakInfo()
MejaBelajar	bahan, jumlahLaci	taruhBarang(), bersihkan(), cetakInfo()

G. Class Demo (Main)

LAMPIRAN 1 – Kode Program

Elektronik.java

```
public class Elektronik {
    private String merk;
    private int tahunProduksi;

    public void setMerk(String merkName) {
        merk = merkName;
    }

    public void setTahunProduksi(int tahun) {
        tahunProduksi = tahun;
    }

    public void nyalakan() {
        System.out.println(merk + " dinyalakan.");
    }

    public void matikan() {
        System.out.println(merk + " dimatikan.");
    }

    public void cetakInfo() {
        System.out.println("Merk : " + merk);
        System.out.println("Tahun Produksi : " + tahunProduksi);
    }
}
```

Laptop.java

```
public class Laptop extends Elektronik {
    private int kapasitasRAM;
    private String jenisProcessor;

    public void setKapasitasRAM(int ram) {
        kapasitasRAM = ram;
    }

    public void setJenisProcessor(String processor) {
        jenisProcessor = processor;
    }

    public void bukaAplikasi(String namaAplikasi) {
        System.out.println("Membuka aplikasi " + namaAplikasi + "...");
    }

    public void installSoftware(String namaSoftware) {
        System.out.println("Menginstall " + namaSoftware + "...");
    }

    @Override
    public void cetakInfo() {
        super.cetakInfo();
        System.out.println("RAM : " + kapasitasRAM + " GB");
        System.out.println("Processor : " + jenisProcessor);
        System.out.println("Jenis Perangkat : Laptop");
    }
}
```

HandPhone.java

```
public class HandPhone extends Elektronik {
    private int kapasitasBaterai;
    private int jumlahKamera;
```

```

    public void setKapasitasBaterai(int baterai) {
        kapasitasBaterai = baterai;
    }

    public void setJumlahKamera(int kamera) {
        jumlahKamera = kamera;
    }

    public void teleponSeseorang(String nama) {
        System.out.println("Menelepon " + nama + "...");
    }

    public void kirimPesan(String nama, String pesan) {
        System.out.println("Mengirim pesan ke " + nama + ": " + pesan);
    }

    @Override
    public void cetakInfo() {
        super.cetakInfo();
        System.out.println("Kapasitas Baterai : " + kapasitasBaterai + " mAh");
        System.out.println("Jumlah Kamera : " + jumlahKamera);
        System.out.println("Jenis Perangkat : HandPhone");
    }
}

```

Kulkas.java

```

public class Kulkas {
    private String merk;
    private int kapasitas;

    public void setMerk(String merkName) {
        merk = merkName;
    }

    public void setKapasitas(int liter) {
        kapasitas = liter;
    }

    public void dinginkan() {
        System.out.println("Kulkas " + merk + " sedang mendinginkan...");
    }

    public void bukaPintu() {
        System.out.println("Pintu kulkas " + merk + " dibuka.");
    }

    public void cetakInfo() {
        System.out.println("Merk : " + merk);
        System.out.println("Kapasitas : " + kapasitas + " liter");
        System.out.println("Jenis Objek : Kulkas");
    }
}

```

MejaBelajar.java

```

public class MejaBelajar {
    private String bahan;
    private int jumlahLaci;

    public void setBahan(String bahanMeja) {
        bahan = bahanMeja;
    }

    public void setJumlahLaci(int laci) {
        jumlahLaci = laci;
    }
}

```

```

public void taruhBarang(String barang) {
    System.out.println(barang + " diletakkan di atas meja belajar.");
}

public void bersihkan() {
    System.out.println("Meja belajar dibersihkan.");
}

public void cetakInfo() {
    System.out.println("Bahan : " + bahan);
    System.out.println("Jumlah Laci : " + jumlahLaci);
    System.out.println("Jenis Objek : Meja Belajar");
}
}

```

Demo.java

```

public class Demo {
    public static void main(String[] args) {

        System.out.println("=== Objek Laptop ===");
        Laptop laptop1 = new Laptop();
        laptop1.setMerk("Asus");
        laptop1.setTahunProduksi(2023);
        laptop1.setKapasitasRAM(16);
        laptop1.setJenisProcessor("Intel Core i7");
        laptop1.nyalakan();
        laptop1.bukaAplikasi("Visual Studio Code");
        laptop1.installSoftware("Java JDK 17");
        laptop1.cetakInfo();
        laptop1.matikan();

        System.out.println();
        System.out.println("=== Objek HandPhone ===");
        HandPhone hp1 = new HandPhone();
        hp1.setMerk("Samsung");
        hp1.setTahunProduksi(2024);
        hp1.setKapasitasBaterai(5000);
        hp1.setJumlahKamera(3);
        hp1.nyalakan();
        hp1.teleponSeseorang("Budi");
        hp1.kirimPesan("Budi", "Halo, apa kabar?");
        hp1.cetakInfo();
        hp1.matikan();

        System.out.println();
        System.out.println("=== Objek Kulkas ===");
        Kulkas kulkas1 = new Kulkas();
        kulkas1.setMerk("LG");
        kulkas1.setKapasitas(300);
        kulkas1.bukaPintu();
        kulkas1.dinginkan();
        kulkas1.cetakInfo();

        System.out.println();
        System.out.println("=== Objek Meja Belajar ===");
        MejaBelajar meja1 = new MejaBelajar();
        meja1.setBahan("Kayu Jati");
        meja1.setJumlahLaci(3);
        meja1.taruhBarang("Buku");
        meja1.bersihkan();
        meja1.cetakInfo();
    }
}

```

LAMPIRAN 2 – Hasil Output Program

Berikut adalah hasil eksekusi program setelah class **Demo** dijalankan:

```
=== Objek Laptop ===
Asus dinyalakan.
Membuka aplikasi Visual Studio Code...
Menginstall Java JDK 17...
Merk : Asus
Tahun Produksi : 2023
RAM : 16 GB
Processor : Intel Core i7
Jenis Perangkat : Laptop
Asus dimatikan.

=== Objek HandPhone ===
Samsung dinyalakan.
Menelepon Budi...
Mengirim pesan ke Budi: Halo, apa kabar?
Merk : Samsung
Tahun Produksi : 2024
Kapasitas Baterai : 5000 mAh
Jumlah Kamera : 3
Jenis Perangkat : HandPhone
Samsung dimatikan.

=== Objek Kulkas ===
Pintu kulkas LG dibuka.
Kulkas LG sedang mendinginkan...
Merk : LG
Kapasitas : 300 liter
Jenis Objek : Kulkas

=== Objek Meja Belajar ===
Buku diletakkan di atas meja belajar.
Meja belajar dibersihkan.
Bahan : Kayu Jati
Jumlah Laci : 3
Jenis Objek : Meja Belajar
```

Output :

```
Problems  Output  Debug Console  Terminal  Source Control  Ports  Run: OOP

• PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Praktik Objek Basis\TugasPraktikum\src> javac *.java
• PS C:\Users\RINGGA BUDI UTAMA\OneDrive\Dokumen\Praktik Objek Basis\TugasPraktikum\src> java Demo

=== Objek Laptop ===
Asus dinyalakan.
Membuka aplikasi Visual Studio Code...
Menginstall Java JDK 17...
Merk : Asus
Tahun Produksi : 2023
RAM : 16 GB
Processor : Intel Core i7
Jenis Perangkat : Laptop
Asus dimatikan.

=== Objek HandPhone ===
Samsung dinyalakan.
Menelepon Budi...
Mengirim pesan ke Budi: Halo, apa kabar?
Merk : Samsung
Tahun Produksi : 2024
Kapasitas Baterai : 5000 mAh
Jumlah Kamera : 3
Jenis Perangkat : HandPhone
Samsung dimatikan.

=== Objek Kulkas ===
Pintu kulkas LG dibuka.
Kulkas LG sedang mendinginkan...
Merk : LG
Kapasitas : 300 liter
Jenis Objek : Kulkas

=== Objek Meja Belajar ===
Buku diletakkan di atas meja belajar.
Meja belajar dibersihkan.
Bahan : Kayu Jati
Jumlah Laci : 3
Jenis Objek : Meja Belajar
```